Proiectul individual: jocuri și cerințe

Sisteme de Operare — Android

Cum alegeți jocul

- Fiecare student alege un singur joc din lista de mai jos.
- Treceți numele, prenumele și jocul ales în tabelul comun: tabelul cu alegerile. Foaia Jocuri din tabel arată câte locuri mai sunt la fiecare joc.
- Același joc poate fi ales de cel mult 4 studenți.
- Variațiile sunt opționale. Fiecare variație funcțională aduce +0,5 puncte la nota proiectului (maximum +1), fără a depăși nota 10. Exemple sunt date la fiecare joc; dacă propuneți altă variație, mi-o trimiteți întâi spre aprobare.
- Aspectul grafic poate fi simplu. Contează să funcționeze regulile, scorul, finalul și Restart.
- De la Laboratorul 5, după fiecare laborator public în so-android-resurse codul complet al Snake-ului pentru acel laborator. Îl folosiți ca exemplu și îl adaptați la jocul vostru; nu îl copiați ca atare.
- Pentru joc creați un proiect Android nou și un repository nou, separat de cel cu Snake. Numele repository-ului: nume-prenume-numejoc (de exemplu popescu-andrei-flappybird).
- Repository-ul este privat și mă adăugați colaborator (alexandra-luca-professor), la fel ca la nume-prenume-labSO.
- Repository-ul nume-prenume-labSO rămâne cu Snake, așa cum este după Laboratorul 4.
- Faceți câte un commit pe fiecare etapă și puneți tag-ul laboratorului la final (lab-05, lab-06 etc.).

Lista jocurilor

Nr.	Joc	Dificultate	Pe scurt
1	Tron solo	ușor	O linie se deplasează pe tablă și lasă urmă în spate. Supraviețuiești cât mai mult.
2	Whack-a-Mole	ușor	O cârtiță apare pentru scurt timp într-o celulă a grilei. Jucătorul o atinge înainte să dispară.
3	Prinde fructele	ușor	Un coș se mișcă stânga–dreapta pe ultimul rând și prinde fructele care cad.
4	Ferește blocurile	ușor	Jucătorul stă pe ultimul rând și evită blocurile care cad de sus.
5	Curse pe 3 benzi	ușor	Mașina jucătorului schimbă banda ca să evite mașinile care vin din față.
6	Labirint cu monede	ușor	Jucătorul se deplasează printr-un labirint, strânge toate monedele și iese.
7	Frogger simplificat	mediu	Broasca traversează benzi de drum cu mașini în mișcare până pe partea cealaltă.
8	Space Shooter simplificat	mediu	Nava jucătorului trage în inamicii care coboară spre ea.
9	Pac-Man simplificat	mediu	Jucătorul mănâncă punctele dintr-un labirint și evită o fantomă.
10	Tetris simplificat	mediu	Piese mici cad pe grilă și se așază una peste alta; rândul complet dispare.
11	Fruit Ninja simplificat	mediu	Fructele sunt aruncate de jos în sus, în arc. Jucătorul le taie atingându-le înainte să cadă.
12	Pong solo	mediu	Mingea ricoșează între pereți, iar jucătorul o respinge cu paleta.

Nr.	Joc	Dificultate	Pe scurt
13	Bomberman simplificat	mediu	Jucătorul pune bombe ca să distrugă cutii și să deschidă drum spre ieșire.
14	Flappy Bird	greu	Pasărea cade sub efectul gravitației și urcă la atingere, trecând prin spațiile dintre obstacole.
15	T-Rex	greu	Dinozaurul aleargă și sare peste obstacolele care vin din față.

Dacă nu vă simțiți încă siguri pe Java, alegeți un joc ușor.

Ce trebuie să aibă proiectul, pe laboratoare

Fiecare laborator adaugă o parte nouă în jocul vostru. În laborator arăt totul pe Snake, iar acasă aplicați același lucru în jocul ales. Pentru Laboratoarele 2–4 puteți porni de la codul vostru de Snake.

Lab	Ce implementați în jocul vostru
Lab 2	Meniul aplicației (titlul jocului, butoanele Start și Instrucțiuni), ecranul jocului și ecranul Instrucțiuni cu regulile jocului vostru. Navigarea cu Intent și revenirea cu finish(), ca la Snake.
Lab 3	Tabla și obiectele jocului desenate pe Canvas, într-un View propriu (ca BoardView). Pozițiile sunt păstrate separat, într-o clasă pentru starea jocului (ca GameState).
Lab 4	Jocul se mișcă și se controlează: butoane sau atingere pentru input, pas periodic cu Handler (update() și apoi invalidate()), oprire la pauză și pornire la revenire, fără accelerare. La jocurile fără pas periodic (Labirint), mutarea se face la fiecare apăsare.
Lab 5	O rundă completă: regulile jocului, scorul afișat pe ecran, finalul și butonul Restart, care pornește o rundă nouă de la zero. Finalul este pierderea; la jocurile care au și victorie (Labirint cu monede, Pac-Man simplificat, Bomberman simplificat) se tratează și victoria.
Lab 6	Recordul (cel mai bun scor) și o setare (de exemplu viteza) se păstrează după închiderea și redeschiderea aplicației. Setarea trebuie să aibă efect în joc.
Lab 7	Istoricul partidelor: fiecare rundă terminată se salvează în baza de date Room (scor și dată) și se afișează într-o listă (RecyclerView), cea mai recentă prima.
Lab 8	Regulile jocului sunt într-o clasă separată (GameEngine), fără cod de desenare. Istoricul se afișează într-un Fragment.
Lab 9	Cel puțin 3 teste JUnit pentru regulile jocului (de exemplu: coliziunea, creșterea scorului, finalul).
Lab 10	Nimic nou de implementat; performanța se discută doar în laborator.
Lab 11	Clasament online cu Firebase: autentificare anonimă, recordul salvat în Firestore și lista celor mai bune scoruri. Jocul trebuie să meargă și fără internet.

Laboratoarele 12 și 13 sunt doar demonstrații; nu adaugă nimic de implementat în proiect.

Ce înseamnă rundă completă

- Jocul pornește la Start și se oprește singur la final.
- Scorul se vede pe ecran în timpul jocului.
- La final apare un mesaj clar (de exemplu „Ai pierdut! Scor: 12”) și butonul Restart.
- Restart resetează tot: pozițiile, scorul, viteza și starea rundei.
- La pauză (Home) jocul se oprește, iar la revenire nu se accelerează.

Descrierea jocurilor

Pentru fiecare joc sunt trecute elementele minime. Puteți adăuga mai mult, dar nu mai puțin.

La fiecare joc, rubrica „Date în GameState” arată ce câmpuri să aibă clasa cu starea jocului, la fel cum Snake are List<Cell> snake și List<Cell> obstacles. De aici porniți codul; în plus, fiecare joc are status (RUNNING / GAME_OVER) și metodele update() și reset(), ca în laborator.

1. Tron solo

O linie se deplasează pe tablă și lasă urmă în spate. Supraviețuiești cât mai mult.

Tabla și elementele: Grilă (de exemplu 20 × 20 celule), desenată pe Canvas. Jucătorul este o celulă colorată; urma rămâne pe celulele parcurse.

Date în GameState (ca lista snake de la Snake): Cell head (capul, ca la Snake); List<Cell> trail (urma, care doar crește, nu se scoate coada); int dx, dy (direcția); int steps (scorul).

Control: 4 butoane de direcție (sus, jos, stânga, dreapta). Nu se poate întoarce direct în sens opus.

Mișcarea: Pas periodic (Handler): la fiecare pas jucătorul avansează o celulă în direcția curentă, iar celula părăsită devine urmă.

Reguli: Coliziune cu marginea tablei sau cu propria urmă = final.

Scor: Numărul de pași supraviețuiți (mai mult = mai bine).

Final: Pierdere la coliziune. Victorie dacă linia umple toată tabla. Victoria este grea: linia trebuie să treacă prin fiecare celulă exact o dată, fără să se lovească de urmă, iar pentru că se mișcă singură, o singură apăsare întârziată înseamnă pierdere. Codul trebuie totuși să trateze victoria.

Setarea (Lab 6): Viteza (de exemplu lent / rapid).

Variații opționale (bonus +0,5 fiecare, maximum +1): obstacole fixe pe tablă; viteza crește la fiecare 50 de pași; un bonus care șterge o parte din urmă.

2. Whack-a-Mole

O cârtiță apare pentru scurt timp într-o celulă a grilei. Jucătorul o atinge înainte să dispară.

Tabla și elementele: Grilă de 3 × 3 sau 4 × 4 găuri, desenată pe Canvas; cârtița este o celulă colorată.

Date în GameState (ca lista snake de la Snake): Cell mole (celula cârtiței, null când nu este vizibilă); int moleTicks (câți pași mai rămâne vizibilă); int score; int missed.

Control: Atingere pe celulă (cellFromTouch din Laboratorul 3).

Mișcarea: Pas periodic: cârtița apare într-o celulă aleatoare (Random), rămâne câțiva pași, apoi dispare și apare în altă parte.

Reguli: Atingerea celulei cu cârtița = lovită (+1) și cârtița dispare. Cârtița care dispare fără să fie atinsă = ratată. Atingerea unei celule goale nu dă puncte.

Scor: Cârtițe lovite (mai mult = mai bine).

Final: Pierdere după 3 cârtițe ratate.

Setarea (Lab 6): Cât timp rămâne cârtița vizibilă.

Variații opționale (bonus +0,5 fiecare, maximum +1): cârtițe aurii care valorează 3 puncte; o bombă care nu trebuie atinsă; două cârtițe în același timp.

3. Prinde fructele

Un coș se mișcă stânga–dreapta pe ultimul rând și prinde fructele care cad.

Tabla și elementele: Grilă; coșul ocupă o celulă (sau două) pe ultimul rând; fructele apar sus în coloane aleatorii.

Date în GameState (ca lista snake de la Snake): Cell basket (coșul, pe ultimul rând); List<Cell> fruits (fructele care cad); int score; int missed.

Control: 2 butoane: stânga și dreapta.

Mișcarea: Pas periodic: fructele coboară un rând; din când în când apare un fruct nou.

Reguli: Fructul ajuns în celula coșului = prins (+1). Fructul ajuns jos fără coș = ratat.

Scor: Fructe prinse (mai mult = mai bine).

Final: Pierdere după 3 fructe ratate.

Setarea (Lab 6): Viteza de cădere.

Variații opționale (bonus +0,5 fiecare, maximum +1): fructe stricate care scad un punct; fructe rare care valorează mai mult; viteza crește în timp.

4. Ferește blocurile

Jucătorul stă pe ultimul rând și evită blocurile care cad de sus.

Tabla și elementele: Grilă; jucătorul pe ultimul rând; blocurile apar sus în coloane aleatorii.

Date în GameState (ca lista snake de la Snake): Cell player (pe ultimul rând); List<Cell> blocks (blocurile care cad); int score.

Control: 2 butoane: stânga și dreapta.

Mișcarea: Pas periodic: blocurile coboară un rând; apar blocuri noi.

Reguli: Bloc în celula jucătorului = coliziune. Fiecare bloc ajuns jos fără coliziune = +1.

Scor: Blocuri evitate (mai mult = mai bine).

Final: Pierdere la coliziune.

Setarea (Lab 6): Viteza de cădere.

Variații opționale (bonus +0,5 fiecare, maximum +1): blocuri late de două celule; viteza crește în timp; o inimă care dă o viață în plus.

5. Curse pe 3 benzi

Mașina jucătorului schimbă banda ca să evite mașinile care vin din față.

Tabla și elementele: 3 benzi verticale (3 coloane); mașina jucătorului jos; celelalte mașini apar sus.

Date în GameState (ca lista snake de la Snake): int playerLane (banda jucătorului: 0, 1 sau 2); List<Cell> cars (col = banda, row = rândul); int score.

Control: 2 butoane: stânga și dreapta (schimbă banda).

Mișcarea: Pas periodic: mașinile din față coboară; apar mașini noi pe benzi aleatorii.

Reguli: Mașina jucătorului și o altă mașină pe aceeași bandă și pe același rând = coliziune.

Scor: Mașini evitate (mai mult = mai bine).

Final: Pierdere la coliziune.

Setarea (Lab 6): Viteza inițială.

Variații opționale (bonus +0,5 fiecare, maximum +1): viteza crește la fiecare 10 mașini evitate; o a patra bandă; monede care pot fi strânse.

6. Labirint cu monede

Jucătorul se deplasează printr-un labirint, strânge toate monedele și iese.

Tabla și elementele: Grilă cu pereți, culoar liber, monede și o celulă de ieșire. Labirintul poate fi definit fix în cod.

Date în GameState (ca lista snake de la Snake): Cell player; List<Cell> walls (pereții, ca obstacolele din Lab 3); List<Cell> coins; Cell exit; int moves (scorul).

Control: 4 butoane de direcție. Fiecare apăsare mută jucătorul o celulă (fără pas periodic).

Mișcarea: Mișcarea are loc doar la apăsare. Se poate adăuga un cronometru care rulează în timpul rundei.

Reguli: Nu se poate intra în perete. Moneda dispare când este strânsă. Ieșirea se deschide doar după toate monedele.

Scor: Numărul de mutări sau timpul până la ieșire (mai puțin = mai bine).

Final: Victorie la ieșire, după strângerea tuturor monedelor. Restart pornește un labirint nou sau același labirint de la început.

Setarea (Lab 6): Dimensiunea labirintului sau dificultatea (de exemplu labirint mic / mare).

Variații opționale (bonus +0,5 fiecare, maximum +1): o limită de mutări; o cheie care deschide o ușă; un al doilea nivel.

7. Frogger simplificat

Broasca traversează benzi de drum cu mașini în mișcare până pe partea cealaltă.

Tabla și elementele: Grilă cu rânduri: rândul de start jos, câteva benzi cu mașini și rândul de sosire sus.

Date în GameState (ca lista snake de la Snake): Cell frog; List<Cell> cars (mașinile, fiecare pe rândul benzii ei); int crossings (traversări reușite).

Control: 4 butoane de direcție; fiecare apăsare mută broasca o celulă.

Mișcarea: Pas periodic: mașinile avansează pe banda lor; când ies de pe tablă, reapar pe partea opusă.

Reguli: Broasca și o mașină în aceeași celulă = coliziune. Ajunsă sus, broasca revine la start și primește un punct.

Scor: Numărul de traversări reușite (mai mult = mai bine).

Final: Pierdere la coliziune (sau după 3 vieți, dacă alegeți vieți).

Setarea (Lab 6): Viteza mașinilor.

Variații opționale (bonus +0,5 fiecare, maximum +1): benzi cu viteze diferite; 3 vieți; viteza crește după fiecare traversare.

8. Space Shooter simplificat

Nava jucătorului trage în inamicii care coboară spre ea.

Tabla și elementele: Grilă; nava pe ultimul rând; inamicii apar sus.

Date în GameState (ca lista snake de la Snake): Cell ship (pe ultimul rând); List<Cell> enemies; Cell bullet (null când nu există glonț); int score.

Control: Butoane stânga, dreapta și Trage.

Mișcarea: Pas periodic: inamicii coboară un rând; gloanțele urcă un rând.

Reguli: Glonț și inamic în aceeași celulă = inamicul dispare (+1). Cel mult un glonț pe ecran la un moment dat (simplificare).

Scor: Inamici distruși (mai mult = mai bine).

Final: Pierdere când un inamic ajunge pe ultimul rând sau lovește nava.

Setarea (Lab 6): Viteza inamicilor.

Variații opționale (bonus +0,5 fiecare, maximum +1): inamici care au nevoie de două lovituri; un bonus de tragere dublă; valuri tot mai rapide.

9. Pac-Man simplificat

Jucătorul mănâncă punctele dintr-un labirint și evită o fantomă.

Tabla și elementele: Grilă cu pereți; un punct în fiecare celulă liberă; jucătorul și o fantomă.

Date în GameState (ca lista snake de la Snake): Cell player; int dx, dy (direcția, ca la Snake); List<Cell> walls; List<Cell> dots (punctele); Cell ghost; int score.

Control: 4 butoane de direcție.

Mișcarea: Pas periodic: jucătorul avansează în direcția curentă (ca la Snake); fantoma se mută aleator într-o celulă liberă vecină.

Reguli: Nu se poate intra în perete. Punctul mâncat dispare (+1). Jucătorul și fantoma în aceeași celulă = final.

Scor: Puncte mâncate (mai mult = mai bine).

Final: Pierdere la întâlnirea cu fantoma. Victorie când toate punctele sunt mâncate.

Setarea (Lab 6): Viteza fantomei.

Variații opționale (bonus +0,5 fiecare, maximum +1): două fantome; un bonus care permite mâncarea fantomei pentru câțiva pași; un al doilea labirint.

10. Tetris simplificat

Piese mici cad pe grilă și se așază una peste alta; rândul complet dispare.

Tabla și elementele: Grilă de 10 coloane × 16 rânduri; celulele ocupate de piesele așezate rămân colorate.

Date în GameState (ca lista snake de la Snake): List<Cell> piece (celulele piesei care cade); boolean[][] filled (celulele ocupate: filled[col][row]); int linesCleared (scorul).

Control: Butoane stânga, dreapta și jos (coborâre mai rapidă).

Mișcarea: Pas periodic: piesa curentă coboară un rând. Când nu mai poate coborî, celulele ei devin ocupate și apare o piesă nouă sus. Piesele sunt de 1 sau 2 pătrate.

Reguli: Piesa nu poate intra în perete sau în celule ocupate. Un rând complet se șterge, iar rândurile de deasupra coboară cu unul.

Scor: Rânduri șterse (mai mult = mai bine).

Final: Pierdere când piesa nouă nu mai are loc sus.

Setarea (Lab 6): Viteza de cădere.

Variații opționale (bonus +0,5 fiecare, maximum +1): piese de 3 pătrate în linie; rotirea piesei; viteza crește la fiecare 5 rânduri șterse.

11. Fruit Ninja simplificat

Fructele sunt aruncate de jos în sus, în arc. Jucătorul le taie atingându-le înainte să cadă.

Tabla și elementele: Canvas; fructele sunt cercuri colorate care apar de sub marginea de jos a ecranului.

Date în GameState (ca lista snake de la Snake): List<Fruit> fruits (clasa Fruit din secțiunea de la final, în loc de Cell); int score; int missed.

Control: Atingere pe fruct = tăiat (nu se cere glisare).

Mișcarea: Pas periodic: gravitația încetinește fructul la urcare și îl accelerează la coborâre; din când în când este aruncat un fruct nou, cu poziție și viteză aleatoare.

Reguli: Atingerea în interiorul cercului taie fructul (+1) și îl elimină. Fructul care cade sub ecran fără să fie atins este ratat.

Scor: Fructe tăiate (mai mult = mai bine).

Final: Pierdere după 3 fructe ratate.

Setarea (Lab 6): Frecvența cu care apar fructele.

Variații opționale (bonus +0,5 fiecare, maximum +1): bombe care termină jocul dacă sunt atinse; fructe mici care valorează mai mult; frecvența crește în timp.

Acest joc folosește mișcare continuă (în pixeli), nu grilă. Vedeți secțiunea „Indicații pentru jocurile cu mișcare continuă” de la final.

12. Pong solo

Mingea ricoșează între pereți, iar jucătorul o respinge cu paleta.

Tabla și elementele: Canvas; pereți sus, stânga și dreapta; paleta jos; mingea.

Date în GameState (ca lista snake de la Snake): Ball ball (din secțiunea de la final); Box paddle (paleta); int returns (scorul).

Control: Butoane stânga–dreapta sau glisare cu degetul.

Mișcarea: Pas periodic: mingea se deplasează și ricoșează din pereți și din paletă.

Reguli: Mingea respinsă de paletă = +1. Mingea care trece de paletă = ratată.

Scor: Retururi reușite (mai mult = mai bine).

Final: Pierdere când mingea este ratată (sau după 3 ratări).

Setarea (Lab 6): Viteza mingii.

Variații opționale (bonus +0,5 fiecare, maximum +1): viteza crește la fiecare 5 retururi; un obstacol în mijlocul terenului; paleta mai mică pe dificultate mare.

Acest joc folosește mișcare continuă (în pixeli), nu grilă. Vedeți secțiunea „Indicații pentru jocurile cu mișcare continuă" de la final.

13. Bomberman simplificat

Jucătorul pune bombe ca să distrugă cutii și să deschidă drum spre ieșire.

Tabla și elementele: Grilă cu pereți ficși, cutii care pot fi distruse și o ieșire ascunsă sub o cutie.

Date în GameState (ca lista snake de la Snake): Cell player; List<Cell> walls (ficși); List<Cell> boxes (care pot fi distruse); Cell exit; Cell bomb (null când nu există bombă); int score.

Control: 4 butoane de direcție și un buton Bombă.

Mișcarea: Jucătorul se mută la apăsare. Bomba explodează după un timp fix (de exemplu 2 secunde, cu Handler).

Reguli: Explozia distruge cutiile vecine (sus, jos, stânga, dreapta). Jucătorul prins în explozie = final. Cel mult o bombă activă.

Scor: Cutii distruse sau timpul până la ieșire (alegeți unul și precizați-l).

Final: Victorie la ieșire. Pierdere dacă jucătorul este prins în explozie.

Setarea (Lab 6): Raza exploziei sau numărul de cutii.

Variații opționale (bonus +0,5 fiecare, maximum +1): un inamic care se mișcă aleator; bonus pentru rază mai mare; limită de timp.

14. Flappy Bird

Pasărea cade sub efectul gravitației și urcă la atingere, trecând prin spațiile dintre obstacole.

Tabla și elementele: Canvas; pasărea într-o poziție fixă pe orizontală; perechi de obstacole (sus și jos, cu un spațiu liber) care vin din dreapta.

Date în GameState (ca lista snake de la Snake): Bird bird (din secțiunea de la final); List<Pipe> pipes; int score.

Control: Atingere pe ecran = salt.

Mișcarea: Pas periodic: viteza verticală crește (gravitație), poziția se actualizează; obstacolele se deplasează spre stânga.

Reguli: Coliziune cu un obstacol, cu solul sau cu marginea de sus = final. Un obstacol depășit = +1.

Scor: Obstacole trecute (mai mult = mai bine).

Final: Pierdere la coliziune.

Setarea (Lab 6): Dificultatea: mărimea spațiului dintre obstacole.

Variații opționale (bonus +0,5 fiecare, maximum +1): obstacole care se mișcă sus–jos; spațiul se micșorează în timp; o monedă între obstacole.

Acest joc folosește mișcare continuă (în pixeli), nu grilă. Vedeți secțiunea „Indicații pentru jocurile cu mișcare continuă" de la final.

15. T-Rex

Dinozaurul aleargă și sare peste obstacolele care vin din față.

Tabla și elementele: Canvas; dinozaurul pe sol, în stânga; obstacolele vin din dreapta.

Date în GameState (ca lista snake de la Snake): Dino dino (din secțiunea de la final); List<Box> obstacles; int score (distanța sau pașii).

Control: Atingere pe ecran = salt (doar când dinozaurul este pe sol).

Mișcarea: Pas periodic: gravitație pentru salt; obstacolele se deplasează spre stânga; apar obstacole noi la intervale variate.

Reguli: Coliziune cu un obstacol = final. Nu se poate sări din aer.

Scor: Distanța sau timpul supraviețuit (mai mult = mai bine).

Final: Pierdere la coliziune.

Setarea (Lab 6): Viteza inițială.

Variații opționale (bonus +0,5 fiecare, maximum +1): obstacole zburătoare peste care nu se sare, ci se trece pe dedesubt; viteza crește în timp.

Acest joc folosește mișcare continuă (în pixeli), nu grilă. Vedeți secțiunea „Indicații pentru jocurile cu mișcare continuă” de la final.

Indicații pentru jocurile cu mișcare continuă

Pentru Flappy Bird, T-Rex, Fruit Ninja simplificat și Pong solo, obiectele nu se mută câte o celulă, ca la Snake, ci câțiva pixeli la fiecare pas. Restul rămâne la fel ca în laborator: GameLoop cu Handler, update() pentru reguli, invalidate() pentru desenare, Restart și pauză.

- Pozițiile și vitezele sunt float, în pixeli.
- Pasul este mai des decât la Snake: de exemplu gameLoop.setIntervalMs(20), adică aproximativ 50 de pași pe secundă, ca mișcarea să pară continuă.
- Valorile din exemple (gravitație, viteză, mărimi) sunt pentru un ecran de telefon obișnuit. Le puteți ajusta, de exemplu ca procent din lățimea tablei.
- Desenarea se face cu aceleași metode din Lab 3: drawRect() pentru dreptunghiuri și drawCircle() pentru minge și fructe.

Coliziunea a două dreptunghiuri

Două dreptunghiuri se ating dacă se suprapun pe ambele axe. Clasa Box este folosită de Flappy Bird, T-Rex și Pong solo. (În Android există și clasa RectF, cu metoda RectF.intersects(), care face același lucru.)

```
public class Box {
    public float left, top, right, bottom;

    public Box(float left, float top, float right, float bottom) {
        this.left = left;
        this.top = top;
        this.right = right;
        this.bottom = bottom;
    }

    public boolean intersects(Box o) {
        return left < o.right && right > o.left
                && top < o.bottom && bottom > o.top;
    }
}
```

Flappy Bird: gravitație și salt

La fiecare pas, gravitația crește viteza în jos, iar viteza schimbă poziția. Saltul pune o viteză negativă (în sus); apoi gravitația o încetinește și pasărea începe să cadă.

```
public class Bird {
    public static final float GRAVITY = 1.2f;
    public static final float JUMP_VELOCITY = -14f;
    public static final float SIZE = 40f;

    public float x = 150f;
    public float y = 400f;
    public float vy = 0f;

    public void jump() {
        vy = JUMP_VELOCITY;
    }

    public void update() {
        vy += GRAVITY;
        y += vy;
    }

    public Box box() {
        return new Box(x, y, x + SIZE, y + SIZE);
    }
}
```

Obstacolul este o pereche de țevi cu un spațiu liber între gapTop și gapBottom. Țeava se mută spre stânga; pasărea pierde dacă atinge una dintre țevi.

```
public class Pipe {
    public static final float WIDTH = 80f;
    public static final float SPEED = 6f;

    public float x;
    public final float gapTop;
    public final float gapBottom;
    public boolean passed = false;

    public Pipe(float x, float gapTop, float gapBottom) {
        this.x = x;
        this.gapTop = gapTop;
        this.gapBottom = gapBottom;
    }

    public void update() {
        x -= SPEED;
    }

    public boolean hits(Box bird) {
        Box top = new Box(x, 0, x + WIDTH, gapTop);
        Box bottom = new Box(x, gapBottom, x + WIDTH, 100000f);
        return bird.intersects(top) || bird.intersects(bottom);
    }
}
```

În update(), punctul se dă o singură dată, când țeava a trecut complet de pasăre:

```
if (!pipe.passed && pipe.x + Pipe.WIDTH < bird.x) {
    pipe.passed = true;
    score++;
}
```

Pierdere: pipe.hits(bird.box()) este true, sau pasărea iese sus (y < 0) ori jos (y + SIZE > înălțimea tablei).

T-Rex: salt doar de pe sol

Ca la Flappy Bird, dar saltul este permis doar când dinozaurul este pe sol, iar la aterizare îl oprim exact pe sol.

```
public class Dino {
    public static final float GRAVITY = 1.5f;
    public static final float JUMP_VELOCITY = -22f;
    public static final float SIZE = 50f;

    public final float groundY;
    public float x = 100f;
    public float y;
    public float vy = 0f;
    public boolean onGround = true;

    public Dino(float groundY) {
        this.groundY = groundY;
        this.y = groundY - SIZE;
    }

    public void jump() {
        if (onGround) {
            vy = JUMP_VELOCITY;
            onGround = false;
        }
    }

    public void update() {
        if (onGround) {
            return;
        }
        vy += GRAVITY;
        y += vy;
        if (y >= groundY - SIZE) {
            y = groundY - SIZE;
            vy = 0f;
            onGround = true;
        }
    }

    public Box box() {
        return new Box(x, y, x + SIZE, y + SIZE);
    }
}
```

Obstacolele sunt dreptunghiuri (Box) care se mută spre stânga; pierdere când dino.box().intersects(obstacol).

Pong solo: mingea și ricoșeul

Mingea are viteză pe ambele axe (vx, vy). La perete schimbăm semnul vitezei pe axa respectivă: vx = -vx pentru pereții laterali, vy = -vy pentru tavan.

```
public class Ball {
    public static final float RADIUS = 15f;

    public float x;
    public float y;
    public float vx;
    public float vy;

    public Ball(float x, float y, float vx, float vy) {
        this.x = x;
        this.y = y;
        this.vx = vx;
        this.vy = vy;
    }

    public void update(float width) {
        x += vx;
        y += vy;

        if (x - RADIUS < 0) {
            x = RADIUS;
            vx = -vx;
        }
        if (x + RADIUS > width) {
            x = width - RADIUS;
            vx = -vx;
        }
        if (y - RADIUS < 0) {
            y = RADIUS;
            vy = -vy;
        }
    }

    public boolean bounceOnPaddle(Box paddle) {
        if (vy > 0 && box().intersects(paddle)) {
            y = paddle.top - RADIUS;
            vy = -Math.abs(vy);
            return true;
        }
        return false;
    }

    public boolean isLost(float height) {
        return y - RADIUS > height;
    }

    public Box box() {
        return new Box(x - RADIUS, y - RADIUS, x + RADIUS, y + RADIUS);
    }
}
```

La Pong solo, fiecare bounceOnPaddle() care întoarce true este un retur reușit (+1). Pierdere: ball.isLost(înălțimea tablei).

Fruit Ninja simplificat: aruncare în arc și atingere

Fructul pornește de sub ecran cu viteză negativă (în sus). Gravitația îl încetinește, apoi îl face să cadă, deci traiectoria este un arc. Fructul este tăiat dacă atingerea este în interiorul cercului: distanța de la atingere la centru este cel mult raza.

```
public class Fruit {
    public static final float GRAVITY = 0.6f;
    public static final float RADIUS = 45f;

    public float x;
    public float y;
    public float vx;
    public float vy;

    public Fruit(float x, float y, float vx, float vy) {
        this.x = x;
        this.y = y;
        this.vx = vx;
        this.vy = vy;
    }

    public void update() {
        vy += GRAVITY;
        x += vx;
        y += vy;
    }

    public boolean isTouched(float touchX, float touchY) {
        float dx = touchX - x;
        float dy = touchY - y;
        return dx * dx + dy * dy <= RADIUS * RADIUS;
    }

    public boolean isMissed(float height) {
        return vy > 0 && y - RADIUS > height;
    }
}
```

Aruncarea unui fruct nou, cu valori aleatoare (random este un obiect Random, ca la hrana din Snake):

```
float startX = 200 + random.nextInt(700);
float vx = -4 + random.nextInt(9);
float vy = -(28 + random.nextInt(7));
fruits.add(new Fruit(startX, boardHeight + Fruit.RADIUS, vx, vy));
```

La atingere (în onTouchEvent, la ACTION_DOWN), parcurgeți lista și eliminați primul fruct pentru care fruit.isTouched(x, y) este true (+1). În update(), fructele pentru care fruit.isMissed(boardHeight) este true se elimină și se numără ca ratate.

Un pas al jocului (exemplu Flappy Bird)

```
public void update() {
    if (status != Status.RUNNING) {
        return;
    }

    bird.update();
    for (Pipe pipe : pipes) {
        pipe.update();
        if (pipe.hits(bird.box())) {
            status = Status.GAME_OVER;
            return;
        }
        if (!pipe.passed && pipe.x + Pipe.WIDTH < bird.x) {
            pipe.passed = true;
            score++;
        }
    }
    if (bird.y < 0 || bird.y + Bird.SIZE > boardHeight) {
        status = Status.GAME_OVER;
    }
}
```

Atingerea ecranului apelează bird.jump(), la fel cum butoanele de la Snake schimbă direcția.

Predarea finală

- Codul complet în repository-ul nume-prenume-numejoc, cu commit-uri pe etape și tag-urile laboratoarelor.
- README cu: jocul ales, variațiile adăugate (dacă există), cum se joacă, ce cerințe sunt îndeplinite și cum ați verificat fiecare cerință.
- Prezentarea proiectului are loc în săptămâna 14.